

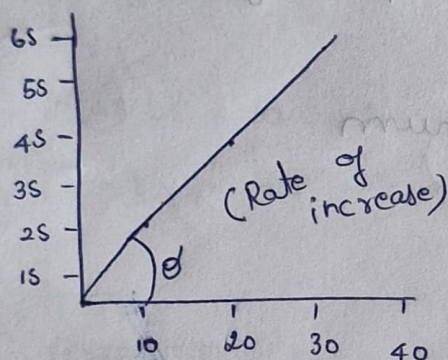
Time and Space Complexity

* TC ! = time taken (it dependent on system config)

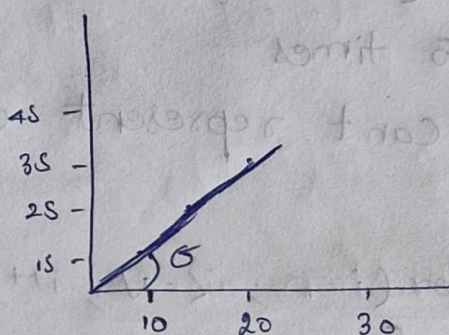
Time Complexity

→ Rate at which the time taken increases with respect to the input

old Window



New Macbook



→ It is computed in Big-O-Notation $O(\downarrow)$

→ It is computed in worst case scenario

time taken
↓
(no. of steps/op involved)

→ Avoid constants

→ Avoid lower values

for($i=1$; $i \leq 5$; $i++$) ①

{ cout << "Raj" } ③

5 (runs 5 time)

Every time $\rightarrow$ It increment, check, print (3)

$$5 \times 3 = 15$$

$O(15) \Rightarrow$ This time Complexity, it run 15 times

$\rightarrow$ Can't represent using num

$\rightarrow$ for($i=1$; $i \leq N$; $i++$)
{ cout << "Raj" }

$$O(N \times 3) \Rightarrow O(3N)$$

$\rightarrow$ Always compute in Worst Case Scenario ✓

$\rightarrow$ Avoid constant

$\rightarrow$ Avoid ^{lower} variable value

$$O(4N^3 + 3N^3) \Rightarrow O(4 \cdot 10^{15} + 3 \cdot 10^{10})$$

$\downarrow$
x

$$O(4N^3 + 3N^3 + 8)$$

$$\downarrow N = 10^5$$

has less significant

```

if (marks < 25) cout << "Grade D"
else if (marks < 45) cout << "grade C"
else if (marks < 65) cout << "grade B"
else cout << "grade A"

```

Best case

when the prgm takes least amt of time

Marks = 10 $O(2)$ [Best case]

Marks = 70 $O(4)$ [Worst case] (Always compute)

Questions:-

```

for (int i = 0; i < N; i++)

```

$i = 0 \rightarrow N$

```

{
  for (int j = 0; j < N; j++)
  {

```

$j = 0 \rightarrow N$

// Block of code (with constant time)

```

}
}

```

Time Complexity

$O(N^2)$

$O(N) \times O(N)$
 → outer loop → inner loop

$i = 0 \rightarrow j = 0 \rightarrow N-1$

$i = 1 \rightarrow j = 0 \rightarrow N-1$

$i = N-1 \rightarrow j = 0 \rightarrow N-1$

Question 2

```
for(i=0; i<N; i++)  
{  
  for(j=0; j<=i; j++)  
  {  
  }  
}
```

$$(1 + 2 + 3 + 4 + \dots + N)$$

$$\frac{N \times (N+1)}{2} = \frac{N^2}{2} + \left(\frac{N}{2}\right) \times$$

$$\Rightarrow O\left(\frac{N^2}{2}\right) \simeq O(N^2)$$

$$1 \quad i=0 \rightarrow \{j=0\}$$

$$2 \quad i=1 \rightarrow \{j=0 \rightarrow 1\}$$

$$3 \quad i=2 \rightarrow \{j=0, 1, 2\}$$

$$n-1 \quad i=n-1 \rightarrow \{j=0 \rightarrow n-1\}$$

Question arises

"The outer loop runs too, so shouldn't we add 1 extra operation for each outer loop execution?"

→ outer loop itself does not perform any additional work. It simply control how many times the inner loop runs

→ The outer loop does not perform extra work, it only control i value

→ The total count of operation is already rep by the num of inner loop execution

i (outer loop) iteration	inner loop (j from 0 to i)	total operation (Including outer + Inner)
i=0	j runs 1 time (0)	1
i=1	" " 2 " (0, 1)	2
i=2	3 (0, 1, 2)	3
i=3	4 (0, 1, 2, 3)	4
i=n-1	n (0, 1, 2, 3, n-1)	n

if we take outer loop also

$$O(n^2) + O(n) \rightarrow \begin{matrix} \text{outer} \\ \text{loop} \end{matrix} \Rightarrow O(n^2 + \underbrace{n}) \rightarrow \begin{matrix} \text{ignore} \\ \text{lower} \\ \text{value} \end{matrix}$$

$\downarrow$
inner loop

$$\therefore O(n^2)$$

Question 3

```
for (int i=0; i<n; i++)
```

```
{ some_operation() }
```

→ runs n time

```
for (int j=0; j<=i; j++)
```

```
{
```

```
another_operation()
```

→ $1+2+3+\dots+(n-1)$
 $O(n^2)$

```
}
```

```
}
```

total work: $O(n) + O(n^2) \Rightarrow O(n^2)$

	outer loop work	inner loop	Total operation
$i=0$	1	1	$1+1=2$
$i=1$	1	2	$1+2=3$
$i=2$	1	3	$1+3=4$
$i=3$	1	4	$1+4=5$
$\vdots$	$\vdots$		
$i=n-1$	1	n	$1+n=n+1$

* total work done by outer loop operation

$$\Rightarrow 1 + 1 + 1 + 1 + \dots + 1 \text{ (n times)} \Rightarrow O(n)$$

* total work done by inner loop operations

$$\Rightarrow 1 + 2 + 3 + 4 + 5 + \dots + n \Rightarrow \frac{n(n+1)}{2} = O(n^2)$$

$$O(n) + O(n^2) \Rightarrow \boxed{O(n^2)}$$

Even though the outer loop does some-operation in each iteration, its effect is negligible compared to inner loop operation

Space Complexity

- Mem space that your prgm takes
- It varies from machine to machine
- Calculates using Big-O-Notation

Auxiliary space + Input space



Space that you
take solve the
prb



space that you take
to store the input

(a) (b) } input space

(c) = a + b

→ auxiliary space

to solve the prb, you are using variable c
(extra space)

$$O(3) \Rightarrow O(1) + O(2)$$

$$\text{int } a[N] \Rightarrow O(N)$$

$$15 \approx 10^8$$

$$2^8 = 2 \times 10^8$$